

RHCSA EX200 — Resumo Dia 07/05

Systemd Timers · Cron · at · Chrony · Tuned · Sysctl · Journald · Processos

PROVA: 14/05

! Armadilhas do Dia

Certo	Errado
daemon-reload ANTES de enable --now	Pular daemon-reload — o systemd não enxerga os arquivos .service/.timer novos
OnBootSec=1m junto com OnUnitActiveSec=2h	Só OnUnitActiveSec sem OnBootSec — o ciclo nunca começa, timer fica inativo
OnCalendar=*-*-* 07:00:00 para horário específico	Confundir com OnCalendar=daily — daily sempre dispara à 00:00
45 0 * * * = 12:45 AM (hora 0 = meia-noite)	Usar hora 24 ou hora 12 por engano no crontab
Criar apenas /etc/cron.allow com o usuário permitido	Criar também /etc/cron.deny — desnecessário, pode causar confusão
renice -n N -p PID (muda processo já rodando)	Confundir com nice (que inicia um processo NOVO com essa niceness)
Editar /etc/sysctl.d/99-arquivo.conf + sysctl -p	Editar /proc/sys/... diretamente — funciona, mas não persiste no reboot

Tabela-Chave — Systemd Timer vs Cron vs at

Característica	Systemd Timer	Cron	at
Tipo de tarefa	Recorrente	Recorrente	Uma única vez
Job perdido (servidor desligado)	Sim (Persistent=true)	Não	Sim (roda ao ligar)
Logs	journalctl	/var/log/cron	email/syslog
Arquivo de config	/etc/systemd/system/	crontab -e	at (comando interativo)
Ativar	systemctl enable --now	automático	atd (service)
Cobrado no EX200	SIM — todos os simulados	SIM	SIM (1 simulado)

BLOCO 1 — Systemd Timers

Um Systemd Timer funciona sempre em par: um arquivo .service define O QUE executar, e um arquivo .timer define QUANDO executar. Ambos ficam em /etc/systemd/system/. A ordem de ativação é sempre: daemon-reload → enable --now. Esse tema aparece em TODOS os 6 simulados — decore o template.

Exemplo completo (S1-Q7): timer a cada 1 minuto — logar a data em um arquivo

```
# PASSO 1: criar o script executável
echo '#!/bin/bash' > /usr/local/bin/system-check.sh
echo 'date >> /var/log/system-check.log' >> /usr/local/bin/system-check.sh
chmod +x /usr/local/bin/system-check.sh

# PASSO 2: criar o .service (Type=oneshot = executa e encerra)
cat > /etc/systemd/system/system-check.service << 'EOF'
[Unit]
Description=System Check Service
[Service]
Type=oneshot
ExecStart=/usr/local/bin/system-check.sh
EOF
```

```
# PASSO 3: criar o .timer
cat > /etc/systemd/system/system-check.timer << 'EOF'
[Unit]
Description=Run System Check every minute
[Timer]
OnCalendar=minutely
Unit=system-check.service
[Install]
WantedBy=timers.target
EOF
```

```
# PASSO 4: ativar (ORDEM OBRIGATÓRIA)
systemctl daemon-reload
systemctl enable --now system-check.timer
systemctl list-timers | grep system-check
```

■ **daemon-reload** ANTES de **enable --now**. Sem essa ordem, o **systemd** não enxerga os arquivos novos.

Exemplo completo (S3-Q5): repetição periódica com OnBootSec + OnUnitActiveSec (a cada 2h)

Para repetição periódica usa-se a combinação **OnBootSec** + **OnUnitActiveSec**. **OnBootSec=1m** define o primeiro disparo (1 min após o boot — inicia o ciclo). **OnUnitActiveSec=2h** define que, após cada execução, espera 2h para disparar de novo.

```
echo '#!/bin/bash' > /usr/local/bin/break-time.sh
echo 'echo "Break Time!" >> /tmp/break.log' >> /usr/local/bin/break-time.sh
chmod +x /usr/local/bin/break-time.sh

cat > /etc/systemd/system/break-time.service << 'EOF'
[Unit]
Description=Break Time Service
[Service]
Type=oneshot
ExecStart=/usr/local/bin/break-time.sh
EOF

cat > /etc/systemd/system/break-time.timer << 'EOF'
[Unit]
Description=Run Break Time every 2 hours
[Timer]
OnBootSec=1m
OnUnitActiveSec=2h
Unit=break-time.service
[Install]
WantedBy=timers.target
EOF

systemctl daemon-reload
systemctl enable --now break-time.timer
```

■ **OnBootSec=1m** é obrigatório quando se usa **OnUnitActiveSec**. Sem ele, o contador de 'desde a última execução' nunca começa e o timer fica inativo.

Variações de OnCalendar já cobradas — mesmo template, muda só o horário:

Simulado	Tarefa	OnCalendar	Observação
S1-Q7	Log a cada 1 min	minutely	= *-*- * *:00
S2-Q16	Deletar temp à meia-noite	daily	+ Persistent=true
S3-Q5	Log a cada 2h	OnBootSec=1m / OnUnitActiveSec=2h	não usa OnCalendar
S4-Q5	Log às 07:00	*-*-* 07:00:00	equivale ao cron: 0 7 * * *
S5-Q16 / S6-Q19	Deletar temp às 16:15	*-*-* 16:15:00	validar com systemd-analyze calendar

■ **Persistent=true** (S2): se o servidor estava desligado no horário agendado, o job roda imediatamente quando o servidor volta. O cron simplesmente pula a execução.

Ferramenta de validação (S5-Q16):

```
systemd-analyze calendar "*-*-* 16:15:00"  
# Saída esperada:  
# Normalized form: *-*-* 16:15:00  
# Next elapse: [data e hora da próxima execução]
```

■ `systemd-analyze calendar` 'expressão' valida a sintaxe do `OnCalendar` e mostra exatamente quando o timer vai disparar — use no exame para evitar erro de digitação.

BLOCO 2 — Cron: crontab e Controle de Acesso

Cada linha do crontab segue o formato: MIN HORA DIA MES DIA_SEMANA COMANDO. O controle de acesso é feito pelos arquivos `/etc/cron.allow` e `/etc/cron.deny`.

Campo	Faixa	Exemplo
MIN	0-59	45 = minuto 45
HORA	0-23	0 = meia-noite, 7 = 07h
DIA	1-31	* = todo dia
MES	1-12	* = todo mês
DIA_SEMANA	0-7 (0=Dom)	* = qualquer

Exemplo completo (S3-Q21): cron para root às 12:45 AM

12:45 AM no formato 12h = 00:45 no formato 24h. Portanto: MIN=45, HORA=0.

```
crontab -e -u root  
# Adicionar a linha abaixo e salvar:  
45 0 * * * find /tmp -type f -empty -delete  
  
# Verificar  
crontab -l -u root
```

■ 45 0 = minuto 45, hora 0. Lembre: meia-noite é hora 0, não hora 24. 12:45 AM = 00:45 = '45 0' no crontab.

Exemplo completo (S2-Q25): permitir cron apenas para o usuário 'tom'

Se `/etc/cron.allow` existe: SOMENTE os usuários listados podem usar cron — os demais são negados automaticamente, sem precisar de `/etc/cron.deny`.

```
# Criar o arquivo com apenas tom  
echo 'tom' > /etc/cron.allow  
cat /etc/cron.allow          # tom  
  
# Testar como outro usuário (deve ser negado)  
su - sam  
crontab -l  
# Saída: You (sam) are not allowed to use this program.
```

■ A presença de `/etc/cron.allow` já é suficiente. NÃO crie `/etc/cron.deny` junto — qualquer usuário fora do `cron.allow` é negado automaticamente.

BLOCO 3 — at: Tarefa Única Agendada

O comando `at` agenda tarefas que rodam uma única vez em horário específico — diferente do `cron` (recorrente). Ideal para ações pontuais. O serviço `atd` precisa estar em execução.

Exemplo completo (S4-Q29): agendar tarefa às 23:30

```
# Garantir que o atd está rodando  
systemctl status atd  
dnf install at -y          # se não estiver instalado  
systemctl enable --now atd
```

```
# Agendar a tarefa
at 23:30
at> touch /var/tmp/late_night_check.txt
at> [Ctrl+D para salvar]
# Saída: job 1 at [data] 23:30

# Verificar a fila
atq
# Saída: 1 [data] 23:30 a root
```

■ *at* = uma vez. *cron* = recorrente. Leia o enunciado com atenção — o exame pode pedir um ou o outro dependendo da necessidade. *atrm N* cancela a tarefa de número *N*.

BLOCO 4 — Chrony / NTP

Chrony é a implementação de NTP usada no RHEL. O daemon *chronyd* sincroniza o relógio do sistema com servidores de tempo. *timedatectl* é a interface de linha de comando para fuso horário e NTP.

Exemplo completo (S1·Q4): timezone America/New_York + Chrony + NTP

```
# PASSO 1: definir o fuso horário
timedatectl set-timezone America/New_York
# Para listar todos: timedatectl list-timezones | grep America

# PASSO 2: instalar e habilitar o Chrony
dnf install chrony -y
systemctl enable --now chronyd

# PASSO 3: ativar sincronização NTP e verificar
timedatectl set-ntp true
timedatectl
# Deve mostrar:
# Time zone: America/New_York
# NTP synchronized: yes
```

A mesma sequência de 4 comandos se repete em todos os simulados — só muda o timezone:

Simulado	Timezone pedido
S1·Q4	America/New_York
S2·Q4	Europe/London
S4·Q6	America/Cancun
S5·Q5	America/Los_Angeles
S6·Q6	America/Chicago

■ *pool.ntp.org iburst* no */etc/chrony.conf* acelera a sincronização inicial — *iburst* envia uma rajada de 8 pacotes ao iniciar, em vez de 1.

BLOCO 5 — Tuned: Perfis de Desempenho

O *tuned* é um daemon que aplica otimizações de kernel automaticamente com base em perfis predefinidos, ajustando CPU, disco, rede e energia.

Perfil	Quando usar
throughput-performance	Servidores de dados, banco de dados — maximizar throughput
virtual-guest	VM rodando dentro de um hypervisor
powersave	Economizar energia — laptops, VMs ociosas
balanced	Desktop/laptop geral — equilíbrio entre performance e energia
accelerator-performance	GPU, HPC, latência ultra-baixa — desabilita power saving states

Exemplo completo (S1-Q17): perfil throughput-performance

```
dnf install tuned -y
systemctl enable --now tuned
tuned-adm profile throughput-performance
tuned-adm active
# Saída: Current active profile: throughput-performance
```

Exemplo — combinando dois perfis / stacking (S3-Q11, S4-Q12, S6-Q12)

O tuned suporta combinar (stackar) múltiplos perfis. O segundo perfil tem prioridade em configurações conflitantes.

```
dnf install tuned -y && systemctl enable --now tuned

# S3-Q11 — VM + economia de energia
tuned-adm profile virtual-guest powersave

# S4-Q12 — VM + latência ultra-baixa
tuned-adm profile virtual-guest accelerator-performance

tuned-adm active
# Saída: Current active profile: virtual-guest powersave
```

■ tuned-adm profile P1 P2 — espaço entre os perfis, não vírgula. O segundo perfil tem prioridade em conflitos.

BLOCO 6 — Sysctl: Parâmetros do Kernel

O sysctl permite ler e modificar parâmetros do kernel em tempo de execução. Alterações em /proc/sys/ são temporárias (perdidas no reboot); para persistência, use arquivos em /etc/sysctl.d/.

Exemplo completo (S2/S3/S4/S5/S6): habilitar packet forwarding IPv4 e IPv6, persistente

```
# PASSO 1: criar arquivo dedicado em /etc/sysctl.d/
vim /etc/sysctl.d/99-forwarding.conf
# Conteúdo do arquivo:
net.ipv4.ip_forward = 1
net.ipv6.conf.all.forwarding = 1

# PASSO 2: aplicar AGORA sem reboot
sysctl -p /etc/sysctl.d/99-forwarding.conf
# OU aplicar todos os arquivos de uma vez:
sysctl --system

# VERIFICAR
sysctl net.ipv4.ip_forward          # net.ipv4.ip_forward = 1
sysctl net.ipv6.conf.all.forwarding # net.ipv6.conf.all.forwarding = 1
```

■ O prefixo numérico (99-) define a ordem de leitura — número maior é lido por último e sobrescreve configurações anteriores. Editar /proc/sys/net/ipv4/ip_forward diretamente funciona, mas NÃO persiste após reboot.

BLOCO 7 — Journald: Logs Persistentes

O journald coleta logs de todos os serviços gerenciados pelo systemd. Por padrão (Storage=auto), os logs ficam em RAM e são perdidos no reboot. Com Storage=persistent, os logs são salvos em disco (/var/log/journal).

Exemplo completo (S1-Q21): journald persistente + filtro sshd do boot atual

```
# PASSO 1: configurar Storage=persistent
vim /etc/systemd/journald.conf
# Em [Journal], alterar/adicionar:
Storage=persistent

# PASSO 2: reiniciar e fazer flush
systemctl restart systemd-journald
journalctl --flush
ls -ld /var/log/journal
```

```
# PASSO 3: exportar logs do sshd do boot atual
journalctl -u sshd -b > /var/log/ssh_boot.log
head /var/log/ssh_boot.log
```

Filtros journalctl mais usados no exame:

Comando	Significado
journalctl -u sshd	Logs apenas do sshd
journalctl -b	Logs do boot atual
journalctl -b -1	Logs do boot anterior
journalctl --since '1 hour ago'	Logs da última hora
journalctl -u sshd -b	sshd + boot atual (combinar filtros)
journalctl --disk-usage	Espaço usado pelos logs

Exemplo completo (S5-Q25): journald persistente com limite de 100MiB

```
vim /etc/systemd/journald.conf
# Em [Journal], adicionar/descomentar:
Storage=persistent
SystemMaxUse=100M

systemctl restart systemd-journald
ls -d /var/log/journal
journalctl --disk-usage
```

■ SystemMaxUse define o espaço máximo total dos logs em disco. Ao atingir o limite, logs antigos são deletados automaticamente (rotação).

BLOCO 8 — Processos: nice, renice, kill, signals

nice e renice controlam a prioridade de CPU de processos. A niceness vai de -20 (maior prioridade) a +19 (menor prioridade); o padrão é 0. Apenas root pode usar valores negativos.

Sinal	Número	Ação
SIGTERM	15	Encerramento gracioso — processo pode limpar recursos
SIGKILL	9	Encerramento forçado — não pode ser ignorado
SIGINT	2	Interrupção (equivalente ao Ctrl+C)

Exemplo completo (S1-Q22): sleep 1000 em background, niceness=5, SIGTERM

```
# PASSO 1: iniciar em background
sleep 1000 &
# Shell imprime ex: [1] 12345 (número do job e PID)

# PASSO 2: encontrar o PID (se não anotou)
pgrep -a sleep
# OU: jobs -l

# PASSO 3: mudar niceness para 5
renice -n 5 -p PID
ps -o pid,ni,comm -p PID      # verificar NI=5

# PASSO 4: enviar SIGTERM e confirmar
kill PID
jobs                          # deve mostrar: [1]+ Terminated
```

■ renice MUDA a niceness de um processo já em execução. nice iniciaria um NOVO processo com essa niceness — não confundir os dois.

Variações já cobradas — mesmo padrão, muda o valor de niceness:

Simulado	Processo	Niceness	Sinal
S1-Q22	sleep 1000	5	SIGTERM
S2-Q30	sleep 5000	15	SIGTERM
S3-Q25	sleep 1000	10	SIGTERM

Exemplo completo (S4-Q23): sleep 10000 em background, SIGINT (sem renice)

```
sleep 10000 &
pgrep -a sleep          # localizar o PID

kill -2 PID
# OU equivalente:
kill -SIGINT PID

jobs                    # Terminated
```

■ nice INICIA um processo com uma niceness. renice MUDA a niceness de um processo já rodando. Confundir os dois é erro clássico no exame.

Tabela Comparativa — Erros Comuns

Situação	Correto ✓	Errado ✗
Ativar timer novo	daemon-reload + enable --now	Só enable --now (arquivo não é lido)
Repetição periódica	OnBootSec=1m + OnUnitActiveSec=2h	Só OnUnitActiveSec (nunca dispara)
Horário fixo no timer	OnCalendar=*-*-* 07:00:00	OnCalendar=daily (sempre 00:00)
Cron às 12:45 AM	45 0 * * *	45 24 * * * (hora 24 não existe)
Restringir cron a 1 usuário	Só /etc/cron.allow com o nome	Criar cron.allow E cron.deny juntos
Mudar prioridade de processo ativo	renice -n N -p PID	nice -n N (inicia processo novo)
Persistência de sysctl	/etc/sysctl.d/99-arquivo.conf + sysctl -p	Editar /proc/sys/ direto (não persiste)
Job perdido com servidor desligado	Persistent=true no .timer	Cron — simplesmente pula a execução

✓ Checklist Final

Systemd Timers

- ☐ Par .service (Type=oneshot + ExecStart) e .timer (OnCalendar + Unit=)
- ☐ daemon-reload ANTES de enable --now — ordem obrigatória
- ☐ systemctl list-timers para ver a próxima execução
- ☐ OnCalendar: minutely, daily, *-*-* 07:00:00, *-*-* 16:15:00
- ☐ OnBootSec=1m + OnUnitActiveSec=2h para repetição a cada 2h
- ☐ Persistent=true para rodar job perdido ao religar o servidor

Cron e at

- ☐ crontab formato: MIN HORA DIA MES DIASEM COMANDO
- ☐ 45 0 * * * = 12:45 AM. 0 7 * * * = 07:00
- ☐ /etc/cron.allow: só os listados podem usar cron
- ☐ at 23:30 → prompt at> → Ctrl+D para salvar. atq para verificar

Chrony / Tuned

- ☐ timedatectl set-timezone + dnf install chrony + enable --now chronyd
- ☐ timedatectl set-ntp true + verificar NTP synchronized: yes
- ☐ tuned-adm profile NOME. Combinar: tuned-adm profile P1 P2

Sysctl / Journald

- ☐ /etc/sysctl.d/99-forwarding.conf + sysctl -p arquivo
- ☐ journald: Storage=persistent + SystemMaxUse=100M em journald.conf
- ☐ journalctl -u sshd -b > /var/log/ssh_boot.log

Processos

- ☐ nice -n N cmd & (iniciar). renice -n N -p PID (modificar)
- ☐ kill = SIGTERM(15). kill -9 = SIGKILL. kill -2 = SIGINT